ПР5

Автоматизированные тесты и безопасность API

Отчёт по практической работе №5

Проект Event Map App

Дисциплина: Проектирование и разработка серверной части приложений

Студент: Фамилия Имя, группа ____

Преподаватель: Самат Илияс Мырзабекулы

Алматы, 2026

Аннотация

Отчёт описывает практическую работу №5 проекта Event Map App: автоматизированные модульные и API-тесты, изоляцию тестовых данных, отчёт о покрытии и проверку типовых уязвимостей API. Приведены чек-лист безопасности, найденная проблема (отсутствие rate limiting) и её исправление.

Содержание

1. Цель работы
2. Структура тестов
3. Модульные тесты
4. API и интеграционные тесты
5. Изоляция тестовых данных
6. Отчёт о покрытии
7. Проверка безопасности API
8. Найденная и исправленная проблема
9. Дополнительный дефект, найденный тестами
10. Как запустить тесты
11. Выводы
12. Использованные файлы

1. Цель работы

Цель ПР5 — покрыть проект автоматическими тестами (модульными и API/интеграционными, включая негативные сценарии) и проверить/устранить основные типовые уязвимости API.

- написать не менее 5 модульных тестов бизнес-логики;
- написать не менее 10 API-тестов с позитивными и негативными сценариями;
- обеспечить изоляцию тестовых данных;
- подключить инструмент покрытия и зафиксировать результат;
- проверить проект по чек-листу безопасности API;
- исправить хотя бы одну реально найденную проблему;
- описать запуск тестов в README.

2. Структура тестов

Тесты разделены по сервисам. Модульные тесты проверяют чистые функции валидации и вспомогательную логику, API-тесты — реальные HTTP-маршруты через штатные test client'ы Django и Flask.

Сервис	Файлы	Что проверяется	Тесто в
users-service	accounts/tests_unit.py	валидация регистрации, JSON, JWT, payload пользователя, rate limiter	14
users-service	accounts/tests.py	регистрация, вход, /auth/me, /users, 401/403/409/429, единые ошибки	14
events-service	tests/test_validation.py	валидация события, координат, даты, категории, query-параметров	12
events-service	tests/test_api.py	CRUD, роли, владелец, JWT, 400/401/403/404	16
attendance-service	tests/test_validation.py	валидация тела отметки участия	6
attendance-service	tests/test_api.py	отметка себя/другого, роли, чтение записей, 401/403/404	10
Итого			72

Таблица 1 — Наборы тестов

3. Модульные тесты

Модульные тесты не обращаются к сети. В users-service используется django.test.SimpleTestCase, который запрещает запросы к БД; в Flask-сервисах проверяются чистые функции из validation.py.

Проверка	Ожидаемый результат
Корректные данные регистрации	список ошибок пуст, значения нормализованы
Неверные username/email/password/role	400 ValidationError, ошибки по каждому полю
Границы длины пароля (7 и 129 символов)	ошибка поля password
Пустое тело, невалидный JSON, JSON-массив	400 BadRequest
JWT: состав claims и подпись	sub, role, iss, exp, jti; чужой секрет отклоняется
Payload пользователя	нет password и password_hash
Rate limiter: лимит, окно, изоляция ключей	после лимита allowed=False, окно освобождается
Валидация события и query-параметров	400 с details для неверных полей
Валидация attendance	400 для пустого event_id и недопустимого status

Таблица 2 — Модульные проверки

4. API и интеграционные тесты

API-тесты вызывают реальные маршруты через test client. Для каждой ключевой операции есть позитивный и негативный сценарий.

Операция	Позитивный сценарий	Негативный сценарий
Регистрация	201, пароль хранится как Argon2-хеш	409 дубликат; 400 неверные поля; 429 лимит
Вход	200, выдан JWT	401 неверный пароль; 429 лимит
Профиль	200 с валидным токеном	401 без токена; 401 невалидный токен
Список пользователей	200 для admin	403 для user
Создание события	201 для organizer/admin	401 без токена; 403 для user; 400 невалидные данные
Чтение события	200 по id	404 для несуществующего id
Изменение события	200 владельцем	403 чужим organizer; 400 неверные поля
Удаление события	204 владельцем или admin	403 чужой organizer; 404 повторно
Отметка участия	201 для себя	401 без токена; 403 за другого; 400 неверный status
Чтение отметок	200 свои или admin	403 чужие для user

Таблица 3 — API-сценарии

5. Изоляция тестовых данных

Сервис	Механизм	Гарантия
users-service	Django TestCase: отдельная тестовая БД SQLite in-memory, откат транзакции после каждого теста	рабочая db.sqlite3 не затрагивается, тесты независимы
events-service	отдельный test_events.db, схема через alembic upgrade head, autouse-фикстура очищает events и event_participants	тесты проходят в любом порядке и повторно
attendance-service	autouse-фикстура очищает in-memory attendance_db до и после теста	нет зависимости от порядка выполнения

Таблица 4 — Изоляция тестов

Тестовые БД и файлы покрытия исключены из Git через .gitignore (.coverage*, .coverage-data/, htmlcov/, .pytest_cache/, *.db).

6. Отчёт о покрытии

Единая команда запуска и отчёта:

```
python -m pip install -r requirements-dev.txt
python scripts/run_tests.py
```

Сервис	Покрытие	Покрыто/всего	Комментарий
users-service	88%	280 / 320	не покрыты часть веток middleware, create_admin, внутренний GET /users/{id}
events-service	82%	498 / 605	0% у неиспользуемых storage.py, repository/participants.py, repository/users.py
attendance-service	93%	130 / 140	не покрыты root/health и часть обработчиков ошибок
Итого	≈85%	908 / 1065	покрытие по трём сервисам

Таблица 5 — Покрытие тестами

Покрытие считается по исходному коду без файлов тестов, миграций и management-команд. Жёсткого порога нет: важнее, что покрыты ключевые пользовательские сценарии и негативные ветки.

7. Проверка безопасности API

Проверялись пункты из задания ПР5 и OWASP API Security Top 10 (2023). Статусы: выполнено / исправлено / не применимо.

№	Проверка	Статус	Что сделано и чем подтверждено
1	Параметризованные запросы вместо конкатенации SQL	выполнено	SQLAlchemy ORM и Django ORM; ручной SQL только SELECT 1; конкатенации пользовательских значений нет
2	API не возвращает лишние поля (хэш пароля)	выполнено	_user_payload возвращает id/username/email/role/created_at; тесты проверяют отсутствие password и argon2
3	Ограничение частоты на входе и регистрации	исправлено	было неограниченно; добавлен rate limiter 10 запросов/60 с на IP, 429 + Retry-After
4	Корректные 401/403 на защищённых маршрутах	выполнено	401 без токена/невалидный/просроченный; 403 при нехватке роли или чужом объекте; покрыто тестами
5	Mass assignment и подмена идентификаторов	выполнено	organizer_id и user_id берутся из JWT; role=admin при регистрации отклоняется
6	Валидация и ограничение размера входных данных	выполнено	limit 1..100, title 2..100, description ≤ 2000, координаты в диапазонах
7	Нет stack trace и секретов в ответах	выполнено	единый JSON ошибок; тест 500 проверяет отсутствие текста исключения; секреты в .env
8	Безопасность JWT	выполнено	HS256, обязательные exp/sub/iss, проверка issuer; чужой секрет и просроченный токен отклоняются

Таблица 6 — Чек-лист безопасности API

8. Найденная и исправленная проблема

8.1. Отсутствие rate limiting

До ПР5 эндпоинты /auth/login и /auth/register принимали неограниченное число запросов. Это позволяло перебирать пароли и массово создавать учётные записи.

После исправления добавлен модуль users-service/accounts/ratelimit.py:

- скользящее окно в памяти процесса;
- ключ «действие + IP», при наличии прокси учитывается X-Forwarded-For;
- лимит по умолчанию — 10 запросов за 60 секунд;
- настройки AUTH_RATE_LIMIT и AUTH_RATE_WINDOW_SECONDS в settings.py и .env.example;
- ответ 429 TooManyRequests и заголовок Retry-After.

Пример ответа при превышении лимита:

```
{
  "error": {
    "code": "TooManyRequests",
    "message": "Слишком много запросов. Попробуйте позже.",
    "details": [{"field": null, "message": "повторите запрос через 43 с"}]
  }
}
```

Проверка: test_register_rate_limit_returns_429 и test_login_rate_limit_returns_429, а также unit-тесты SlidingWindowLimiter. Ограничение in-memory и не общее для нескольких инстансов; для production нужен Redis или лимит на reverse proxy.

9. Дополнительный дефект, найденный тестами

При изменении категории события services.py присваивал только event.category_id, а _serialize() читал уже загруженную связь event.category. В ответе оставалась старая категория, хотя в БД сохранялась новая.

Исправление — присваивание связи в replace() и patch():

```
event.category = category
```

Регрессия закрыта тестом test_patch_valid_and_invalid: он проверяет не только статус 200, но и значение поля category.

10. Как запустить тесты

Все сервисы сразу, с отчётом о покрытии:

```
python -m pip install -r requirements-dev.txt
python scripts/run_tests.py
```

Или по отдельности:

```
cd users-service
python manage.py test

cd events-service
python -m pytest -q

cd attendance-service
python -m pytest -q
```

Данные coverage сохраняются в .coverage-data/, отчёт печатается в консоль командой coverage report -m.

11. Выводы

В ПР5 добавлено 72 автоматических теста: 32 модульных и 40 API/интеграционных. Тесты покрывают валидацию, CRUD, регистрацию и вход, роли, проверку владельца и негативные сценарии 400/401/403/404/409/429.

Тестовые данные изолированы, покрытие по трём сервисам составляет около 85%. По чек-листу безопасности найдена и устранена реальная проблема — отсутствие ограничения частоты запросов на аутентификации. Дополнительно тесты выявили и закрыли дефект сериализации категории после PATCH/PUT.

Остаточные ограничения: rate limiter хранит счётчики в памяти процесса, нет отзыва JWT и refresh-токенов, attendance-service хранит данные в памяти. Эти пункты вынесены в следующие работы.

12. Использованные файлы

- users-service/accounts/tests.py, tests_unit.py, ratelimit.py, errors.py, middleware.py, views.py;
- users-service/config/settings.py, .env.example;
- events-service/tests/conftest.py, test_validation.py, test_api.py;
- events-service/services.py, validation.py;
- attendance-service/tests/conftest.py, test_validation.py, test_api.py;
- attendance-service/validation.py, app.py;
- scripts/run_tests.py, requirements-dev.txt;
- README.md, docs/testing.md, docs/security.md.